

در ابتدای پروژه کتابخانه های مورد نیاز را به پروژه اضافه میکنیم.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from collections import Counter
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.metrics import accuracy_score
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import GridSearchCV
```

KNN Implementation

این کلاس برای پیاده سازی الگوریتم KNN طراحی شده است. الگوریتم KNN یک الگوریتم یادگیری ماشین است که در آن پیش بینی بر اساس نزدیکترین همسایگان داده ها انجام می شود.

در تابع **--init--** این متد برای مقداردهی اولیه به پارامترهای کلاس استفاده می شود. k تعداد همسایگانی که برای پیش بینی انتخاب می شود. X_{train} , y_{train} داده های آموزشی که در آن ویژگی ها (X_{train}) و برچسب ها یا کلاس ها (y_{train}) ذخیره می شوند.

در تابع **fit** داده های آموزشی را به مدل می دهد. X ویژگی ها که برای آموزش مدل استفاده می شود. y برچسب ها (خروجی ها) که به عنوان هدف برای پیش بینی قرار می گیرند. در این متد، داده های ورودی (X) و برچسب ها (y) به صورت پارامترهای ورودی به متد **fit** می شوند و درون متغیرهای **self.X_train** و **self.y_train** ذخیره می شوند.

در تابع **predict** پیش بینی های مدل را برای داده های ورودی انجام می دهد. ورودی این متد، داده های جدیدی است که مدل باید برای آن ها پیش بینی انجام دهد.

در تابع **predict_single_point** برای پیش بینی یک نمونه واحد (یک داده) طراحی شده است. در ابتدا، فاصله هر نقطه داده آموزشی از نقطه پیش بینی شده X محاسبه می شود. این فاصله با استفاده از فرمول فاصله اقلیدسی محاسبه می شود سپس، از بین داده های آموزشی، نزدیکترین همسایگان انتخاب می شوند. پس از آن، برچسب های این همسایگان جمع آوری شده و برچسبی که بیشترین تکرار را دارد به عنوان پیش بینی برای نمونه ورودی انتخاب می شود.

```

class KNNClassifier:
    def __init__(self, k):
        self.k = k
        self.X_train = None
        self.y_train = None
    def fit(self, X, y):
        self.X_train = X
        self.y_train = y
    def predict(self, X):
        predictions = [self._predict_single_point(x) for x in X]
        return np.array(predictions)
    def _predict_single_point(self, x):
        distance = np.sqrt(np.sum((self.X_train - x)**2, axis=1))
        knn_indice = np.argsort(distance)[:self.k]
        knn_label = [self.y_train[i] for i in knn_indice]
        most_common = Counter(knn_label).most_common(1)
        return most_common[0][0]

```

Load and preprocess

در ابتدا فایل داده شده را در پارامتر loan_sub ذخیره می‌کنیم. سپس لیستی از ویژگی‌ها که برای مدل استفاده خواهند شد، در search_params قرار دارد و داده‌ها تنها به این ستون‌ها محدود می‌شوند. با استفاده از dropna(subset=['bad_loans'])، سطرهایی که مقدار bad_loans آن‌ها گم شده است، حذف می‌شوند. برای ویژگی‌های غیر عددی مثل term، grade، home_ownership، emp_length، از LabelEncoder برای تبدیل این مقادیر به اعداد استفاده می‌شود. این کار به مدل کمک می‌کند که با داده‌های عددی کار کند. سپس X به ویژگی‌ها همه ستون‌ها به جز bad_loans و y به برچسب‌ها bad_loans اختصاص می‌یابد. داده‌ها به سه مجموعه تقسیم می‌شوند برای آموزش مدل، اعتبارسنجی مدل و آرمایش نهایی مدل و تقسیم‌بندی به طوری انجام می‌شود که نسبت توزیع برچسب‌ها در هر مجموعه مشابه است. از MinMaxScaler برای مقیاس‌بندی داده‌ها به بازه [0, 1] استفاده می‌شود. این کار برای جلوگیری از تأثیر مقیاس‌های مختلف ویژگی‌ها بر مدل ضروری است. همچنین برای مقابله با عدم تعادل در داده‌های آموزشی (که در آن تعداد نمونه‌های کلاس‌های مختلف متفاوت است)، از تکنیک SMOTE برای تولید نمونه‌های مصنوعی از کلاس کمتر دیده‌شده استفاده می‌شود. در نهایت، داده‌های مقیاس‌بندی‌شده و برچسب‌های کلاس‌های آموزشی، اعتبارسنجی و آرمایشی به همراه ویژگی‌های X (اسامی ستون‌ها) بازگشت داده می‌شوند. به طور کلی هدف این قسمت آماده‌سازی داده‌ها جهت استفاده در مدل‌های یادگیری ماشین است.

```
def load_and_preprocess_data(path):
    loan_sub = pd.read_csv(path)

    search_params = ['grade', 'term', 'home_ownership', 'emp_length', 'bad_loans']
    loan_sub = loan_sub[search_params]

    loan_sub = loan_sub.dropna(subset=['bad_loans'])

    for col in ['grade', 'term', 'home_ownership', 'emp_length']:
        loan_sub[col] = LabelEncoder().fit_transform(loan_sub[col])

    X = loan_sub.drop('bad_loans', axis=1)
    y = loan_sub['bad_loans']

    train_x, temp_x, train_y, temp_y = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
    value_x, test_x, value_y, test_y = train_test_split(temp_x, temp_y, test_size=0.5, random_state=42, stratify=temp_y)

    scaler = MinMaxScaler()
    X_train_scaled = scaler.fit_transform(train_x)
    X_val_scaled = scaler.transform(value_x)
    X_test_scaled = scaler.transform(test_x)

    smote = SMOTE(random_state=42)
    X_train_scaled, train_y = smote.fit_resample(X_train_scaled, train_y)

    return X_train_scaled, X_val_scaled, X_test_scaled, train_y, value_y, test_y, X.columns
```

Training functions

در این کد، چهار مدل مختلف یادگیری ماشین برای آموزش داده‌ها تعریف شده است: AdaBoost ، KNN ، DT و Random Forest .

تابع **train_decision_tree** یک مدل درخت تصمیم **DecisionTreeClassifier** را با عمق حداکثر $\text{max_depth}=d$ که به عنوان ورودی از کاربر گرفته می‌شود، ایجاد می‌کند. سپس مدل بر اساس داده‌های ورودی $(X_{\text{train}}, y_{\text{train}})$ آموزش داده می‌شود. در نهایت، مدل آموزش‌دیده برمی‌گردد. این مدل برای حل مسائل طبقه‌بندی استفاده می‌شود و به طور خاص از درخت‌های تصمیم برای تقسیم داده‌ها به خوشه‌های مختلف استفاده می‌کند.

تابع **train_knn** یک مدل k -نزدیک‌ترین همسایه (**KNeighborsClassifier**) ایجاد می‌کند. تعداد همسایه‌ها (k) که در این مدل باید در نظر گرفته شود، به عنوان ورودی از کاربر دریافت می‌شود. سپس مدل بر اساس داده‌های ورودی $(X_{\text{train}}, y_{\text{train}})$ آموزش داده می‌شود. در نهایت، مدل آموزش‌دیده برگردانده می‌شود.

تابع **train_adaboost** یک مدل **AdaBoostClassifier** ایجاد می‌کند. تعداد $n_{\text{estimators}}=n$ ، که تعداد تکرارهای الگوریتم تقویت کننده است، به عنوان ورودی از کاربر گرفته می‌شود. مدل سپس با داده‌های ورودی $(X_{\text{train}}, y_{\text{train}})$ آموزش داده می‌شود. در نهایت، مدل آموزش‌دیده برگردانده می‌شود.

تابع **train_rf** این تابع یک مدل **RandomForestClassifier** را ایجاد می‌کند. ابتدا یک دیکشنری از پارامترهای مدل (**param_grid**) تعریف می‌شود که شامل تعداد درخت‌ها ($n_{\text{estimators}}$)، عمق حداکثر درخت‌ها (**max_depth**)، حداقل تعداد نمونه‌های مورد نیاز برای تقسیم داده‌ها (**min_samples_split**)، ویژگی‌هایی که باید در هر درخت استفاده شوند (**max_features**)، و اینکه آیا از بوت‌استرپ استفاده شود یا خیر (**bootstrap**) است.

سپس مدل جنگل تصادفی ساخته می‌شود و برای جستجوی بهترین پارامترها از روش جستجوی شبکه‌ای (GridSearchCV) استفاده می‌شود. این روش پارامترهای مختلف را تست می‌کند تا بهترین مدل را پیدا کند.

مدل بهترین انتخاب شده بر اساس دقت (scoring='accuracy') به کاربر باز می‌گردد.

جنگل تصادفی یک الگوریتم یادگیری جمعی است که چندین درخت تصمیم را به کار می‌گیرد و پیش‌بینی‌ها را به صورت میانگین از درخت‌های مختلف انجام می‌دهد.

در نهایت، هر یک از این توابع مدل‌های مختلف یادگیری ماشین را آموزش می‌دهند و مدل‌های آموزش‌دیده شده را برای استفاده‌های بعدی باز می‌گردانند.

```
def train_decision_tree(X_train, y_train, d):
    model = DecisionTreeClassifier(max_depth=d, random_state=42)
    model.fit(X_train, y_train)
    return model

def train_knn(X_train, y_train, k):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    return model

def train_adaboost(X_train, y_train, n):
    model = AdaBoostClassifier(n_estimators=n)
    model.fit(X_train, y_train)
    return model

def train_rf(X_train, y_train):
    param_grid = {
        'n_estimators': [50, 100],
        'max_depth': [10, 20],
        'min_samples_split': [2],
        'max_features': ['sqrt', 'log2'],
        'bootstrap': [True]
    }
    rf_model = RandomForestClassifier(random_state=42)
    grid_search = GridSearchCV(rf_model, param_grid, cv=3, n_jobs=-1, scoring='accuracy')
    grid_search.fit(X_train, y_train)
    return grid_search.best_estimator_
```

Main

این بخش هدف آن آموزش و ارزیابی چهار مدل مختلف یادگیری ماشین (درخت تصمیم، k-نزدیکترین همسایه، آدا بوست و جنگل تصادفی) برای پیش‌بینی یک برچسب bad_loans است. ابتدا فایل داده‌ها با استفاده از تابع load_and_preprocess_data بارگذاری و پیش‌پردازش می‌شود. این تابع داده‌ها را به ویژگی‌ها و برچسب‌ها تقسیم کرده و داده‌ها را به مجموعه‌های آموزشی، اعتبارسنجی و آزمایشی تقسیم می‌کند. آموزش مدل درخت تصمیم برای هر عمق درخت تصمیم (از 1 تا 10)، یک مدل درخت تصمیم آموزش داده می‌شود. مدل‌ها بر اساس داده‌های آموزشی آموزش داده شده و سپس دقت آن‌ها بر روی داده‌های اعتبارسنجی ارزیابی می‌شود. مدل با بهترین دقت اعتبارسنجی انتخاب می‌شود. آموزش مدل-k نزدیکترین همسایه (KNN) برای هر تعداد همسایه (k) از 1 تا 20، یک مدل-k نزدیکترین همسایه آموزش داده می‌شود. دقت اعتبارسنجی مدل‌ها محاسبه شده و بهترین مدل انتخاب می‌شود. آموزش مدل (AdaBoost) برای هر تعداد استیمیتور (n) از 10 تا 100 با گام 10، یک مدل آدا بوست آموزش داده می‌شود. دقت اعتبارسنجی مدل‌ها بررسی و بهترین مدل انتخاب می‌شود. آموزش مدل Random Forest جنگل تصادفی با استفاده از بهترین پارامترهای ممکن (که از

طریق جستجوی شبکه‌ای پیدا می‌شود) آموزش داده می‌شود. دقت اعتبارسنجی این مدل نیز محاسبه و ذخیره می‌شود. ارزیابی نهایی مدل‌ها بر روی داده‌های آزمایشی پس از انتخاب بهترین مدل از هر الگوریتم (درخت تصمیم، kNN، آدا بوست و جنگل تصادفی)، دقت نهایی آن‌ها بر روی مجموعه داده‌های آزمایشی (X_test, y_test) محاسبه می‌شود. دقت هر یک از مدل‌ها چاپ می‌شود. مقایسه مدل‌ها تابع `compare_models` که به احتمال زیاد برای مقایسه و انتخاب بهترین مدل نوشته شده است، اجرا می‌شود. این تابع ممکن است دقت هر مدل را نمایش دهد یا مقایسه‌ای گرافیکی انجام دهد. نمایش درخت تصمیم در نهایت، در صورتی که مدل درخت تصمیم انتخاب شده باشد، درخت تصمیم آن با استفاده از `plot_tree` رسم می‌شود. این درخت ویژگی‌ها و پیش‌بینی‌های مدل را به صورت گرافیکی نمایش می‌دهد.

```
def main():
    path = r"D:\Artificial Intelligence\Project\load_sub.csv"
    X_train, X_val, X_test, y_train, y_val, y_test, columns = load_and_preprocess_data(path)

    best_dt_model = None
    best_dt_acc = 0
    for depth in range(1, 11):
        dt_model = train_decision_tree(X_train, y_train, depth)
        val_pred = dt_model.predict(X_val)
        val_acc = accuracy_score(y_val, val_pred)
        if val_acc > best_dt_acc:
            best_dt_acc = val_acc
            best_dt_model = dt_model

    best_knn_model = None
    best_knn_acc = 0
    for k in range(1, 21):
        knn_model = train_knn(X_train, y_train, k=k)
        val_pred = knn_model.predict(X_val)
        val_acc = accuracy_score(y_val, val_pred)
        if val_acc > best_knn_acc:
            best_knn_acc = val_acc
            best_knn_model = knn_model

    best_ab_model = None
    best_ab_acc = 0
    for n in range(10, 110, 10):
        ab_model = train_adaboost(X_train, y_train, n)
        val_pred = ab_model.predict(X_val)
        val_acc = accuracy_score(y_val, val_pred)
        if val_acc > best_ab_acc:
            best_ab_acc = val_acc
            best_ab_model = ab_model

    best_rf_model = None
    best_rf_acc = 0
    rf_model = train_rf(X_train, y_train)
    val_pred = rf_model.predict(X_val)
    val_acc = accuracy_score(y_val, val_pred)
    best_rf_acc = val_acc
    best_rf_model = rf_model

    dt_test_acc = accuracy_score(y_test, best_dt_model.predict(X_test))
    knn_test_acc = accuracy_score(y_test, best_knn_model.predict(X_test))
    ab_test_acc = accuracy_score(y_test, best_ab_model.predict(X_test))
    rf_test_acc = accuracy_score(y_test, best_rf_model.predict(X_test))

    print(f"DT Accuracy: {dt_test_acc:.4f}")
    print(f"kNN Accuracy: {knn_test_acc:.4f}")
    print(f"AdaBoost Accuracy: {ab_test_acc:.4f}")
    print(f"RF Accuracy: {rf_test_acc:.4f}")
    compare_models(dt_test_acc, knn_test_acc, ab_test_acc, rf_test_acc)

    plt.figure(figsize=(60, 30))
    plot_tree(best_dt_model,
              feature_names=columns,
              class_names=["0", "1"],
              filled=True,
              rounded=True,
              fontsize=8
              )
    plt.show()

if __name__ == "__main__":
    main()
```

نتایج:

